

Moore and Mealy FSM

## Question 01:

What are Moore and Mealy FSM in logic design

ChatGPT said:

In logic design and digital systems, **Moore** and **Mealy** are two fundamental models of **finite state machines (FSMs)**. They describe how the **outputs** of the system depend on its **inputs** and **states**.

---

## ★ Moore Machine

### Definition

A **Moore FSM** is a state machine where **outputs depend only on the current state**, *not directly on the inputs*.

### Key Characteristics

- Output changes **only when the state changes**.
- Typically has **more states** than a Mealy machine.
- Outputs are **stable** and not affected by transient input glitches.
- Easier to design and debug.

### Output equation

$\text{Output} = f(\text{State})$

### Diagram style

States have outputs inside them:

(State / Output)

### Example

If a state S1 outputs HIGH (1), it will remain HIGH as long as the machine stays in S1, regardless of input.

# ★ Mealy Machine

## Definition

A **Mealy FSM** is a state machine where **outputs depend on both the current state and the current inputs**.

## Key Characteristics

- Output can change **immediately** when an input changes (without waiting for a state change).
- Typically requires **fewer states** than Moore.
- Reacts faster to inputs.
- Output can be less stable (susceptible to glitches).

## Output equation

$\text{Output} = f(\text{State}, \text{Input})$

## Diagram style

Outputs are shown on transitions:

StateA --input/output--> StateB

## Example

If input changes, output may change instantly even if the state stays the same.



## Quick Comparison

| Feature             | Moore FSM               | Mealy FSM             |
|---------------------|-------------------------|-----------------------|
| Output depends on   | State                   | State + Input         |
| Output changes      | On clock (state change) | Anytime input changes |
| Glitch risk         | Low                     | Higher                |
| Number of states    | More                    | Fewer                 |
| Response time       | One clock cycle later   | Immediate             |
| Hardware complexity | Easier                  | Harder                |



## Intuition

- **Moore = stable output = output tied to states.**
  - **Mealy = responsive machine = output tied to state + input.**
- 

## Simple Visual Example

### **Moore:**

S0 (Y=0) ----> S1 (Y=1)

Output stays 0 in S0 and 1 in S1.

---

### **Mealy:**

S0 --X=1/Y=1--> S1

If input X = 1, output Y becomes 1 immediately, even before reaching S1.

---

--\*--\*--\*--

## Question 02:

can you give state diagram conversions

ChatGPT said:

Sure! Here are clear **Moore  $\leftrightarrow$  Mealy FSM conversion examples** with diagrams and explanation.

---

### 1. Mealy $\rightarrow$ Moore Conversion (Example)

#### Given Mealy Machine

Transitions include outputs:

```
      (input/output)
S0 --0/0--> S0
S0 --1/1--> S1
S1 --0/0--> S0
S1 --1/0--> S1
```

#### Key principle

A Moore machine must have outputs **inside states only**, so if a Mealy state produces **different outputs for different transitions**, we must **split that state into multiple Moore states**.

#### Step-by-step

##### Step 1 — Label Mealy output behavior

Example:

From S0:

- For input 0  $\rightarrow$  output 0
- For input 1  $\rightarrow$  output 1  
 $\rightarrow$  S0 needs to be split into two Moore states.

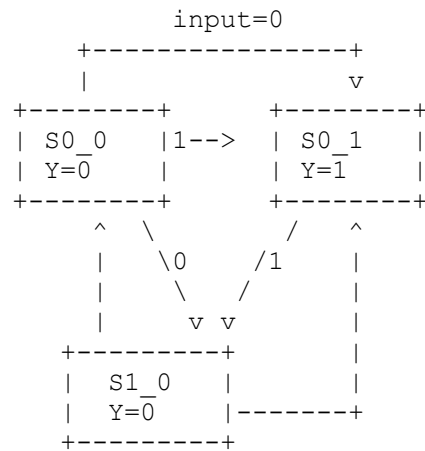
##### Step 2 — Create equivalent Moore states

Make new states based on output:

- S0\_0 : output = 0
- S0\_1 : output = 1

- S1\_0 : output = 0
- (No S1\_1 needed because S1 never outputs 1)

### Step 3 — Draw Moore state diagram



(This is a conceptual layout; transitions follow original Mealy behavior.)

### Outcome

- All outputs now tied to states.
- Equivalent behavior but may have **more states**.

## ✓ 2. Moore → Mealy Conversion (Example)

### Given Moore Machine

Outputs inside states:

S0 (Y=0)

S1 (Y=1)

Transitions:

S0 --0--> S0

S0 --1--> S1

S1 --0--> S0

S1 --1--> S1

### Goal

Move output from states → transitions.

## Step-by-step

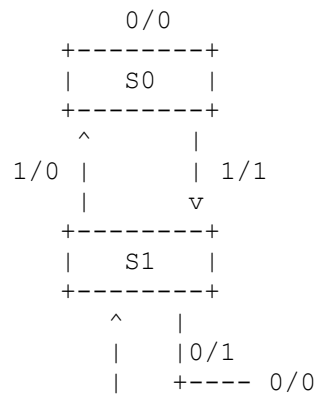
### Step 1 — Place state output on *incoming* transitions

Example:

State S1 has output 1

→ Any transition leading **into S1** gives output 1.

### Step 2 — Draw equivalent Mealy diagram



## Explanation

- Transition into S1 outputs 1 (since S1 has Y=1)
- Transition into S0 outputs 0 (since S0 has Y=0)

## Outcome

- Fewer states than Moore.
- Output can change immediately on input change.

# ★ Summary Table

| Converting    | What changes                                    | What increases     |
|---------------|-------------------------------------------------|--------------------|
| Mealy → Moore | Outputs move from transitions → states          | Number of states ↑ |
| Moore → Mealy | Outputs move from states → incoming transitions | Responsiveness ↑   |

--\*--\*--\*--

## Question 03:

Can you please give a conversion practice problem

ChatGPT said:

Sure! Here is a **clean, exam-style Moore  $\leftrightarrow$  Mealy conversion practice problem** for you to solve.

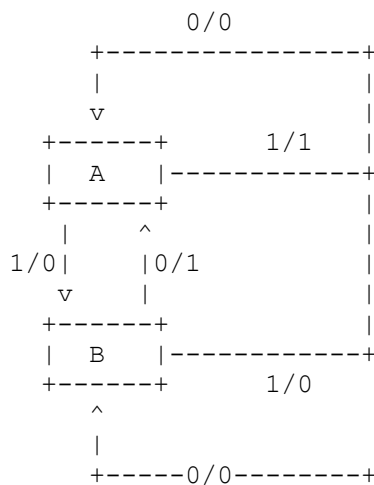
If you want, I can also provide the full step-by-step solution afterward.

---

## ★ Practice Problem: Mealy $\rightarrow$ Moore Conversion

You are given the following **Mealy Machine**:

### State Diagram



### Description of behavior

- In **state A**:
  - Input 0  $\rightarrow$  Output 0, go to A
  - Input 1  $\rightarrow$  Output 1, go to B
- In **state B**:
  - Input 0  $\rightarrow$  Output 1, go to A
  - Input 1  $\rightarrow$  Output 0, go to B

### Your task

1. **Convert this into an equivalent Moore Machine.**
  2. This will require **state splitting** (because some states have different outputs for different inputs).
  3. Draw the new **Moore state diagram**, where **each state has a fixed output**.
- 

## **What you should produce**

- List of new Moore states (with output values)
  - Complete Moore state diagram (transitions labeled only with inputs)
  - Explanation of how you determined the number of states
- 

--\*--\*--\*--



## Question 04:

You said:

Can you provide a sample Moore FSM implementation using VHDL

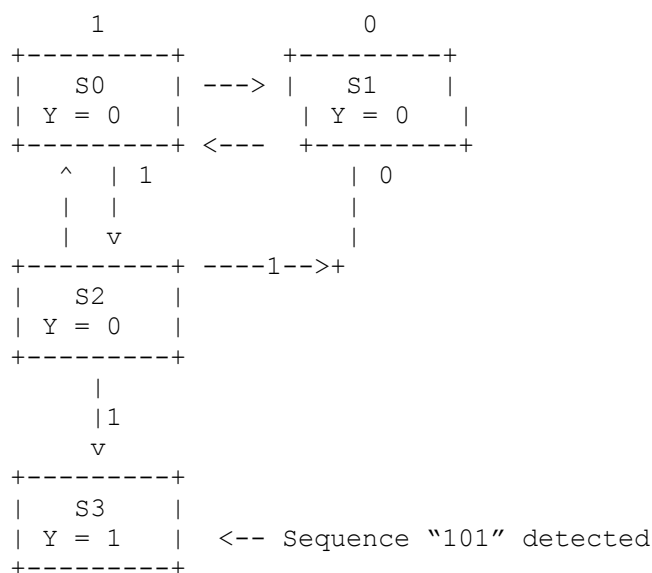
ChatGPT said:

Sure! Here is a **clean, synthesizable VHDL example** of a **Moore FSM**.  
This example implements a simple FSM that detects the sequence **"101"** on an input and outputs 1 **only in the final state**.



## Moore FSM — VHDL Example

### State Diagram (Conceptual)



## VHDL Code: Moore FSM (Sequence Detector)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity moore_fsm is
```

```
Port (
    clk    : in  STD_LOGIC;
    reset  : in  STD_LOGIC;
    x      : in  STD_LOGIC;      -- input bit stream
    y      : out STD_LOGIC      -- Moore output
);
end moore_fsm;

architecture Behavioral of moore_fsm is

    -- State Encoding
    type state_type is (S0, S1, S2, S3);
    signal current_state, next_state : state_type;

begin

    -----
    -- 1. State Register (clocked)
    -----
    process(clk, reset)
    begin
        if reset = '1' then
            current_state <= S0;
        elsif rising_edge(clk) then
            current_state <= next_state;
        end if;
    end process;

    -----
    -- 2. Next State Logic (combinational)
    -----
    process(current_state, x)
    begin
        case current_state is

            when S0 =>
                if x = '1' then
                    next_state <= S1;
                else
                    next_state <= S0;
                end if;

            when S1 =>
                if x = '0' then
                    next_state <= S2;
                else
                    next_state <= S1;
                end if;

            when S2 =>
                if x = '1' then
                    next_state <= S3;
                else
                    next_state <= S0;
                end if;

            when S3 => -- final state
```

```
        if x = '1' then
            next_state <= S1;
        else
            next_state <= S2;
        end if;

    end case;
end process;

-----
-- 3. Output Logic (Moore: depends only on current_state)
-----
y <= '1' when current_state = S3 else '0';

end Behavioral;
```

---



## Why this is a Moore Machine

- The **output y** depends **ONLY on current\_state**, not on input x.
- Output changes *only at clock edges* when the FSM changes state.
- All combinational logic goes into the “next state logic” block.
- The output process is *pure* (no inputs other than the state).

---

--\*--\*--\*--

### Question 05:

can you provide test bench in VHDL for this Moore FSM

ChatGPT said:

Absolutely — here is a **clean, synthesizable, simulation-ready VHDL testbench** for the Moore FSM I gave you earlier (the 101 sequence detector).

This testbench:

- Generates a clock
- Applies reset
- Feeds an input bitstream (test vector)
- Prints waveform-friendly timing
- Shows when the FSM output becomes 1



## VHDL Testbench for the Moore FSM

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_moore_fsm is
end tb_moore_fsm;

architecture behavior of tb_moore_fsm is

    -- Component Declaration for the Unit Under Test (UUT)
    component moore_fsm
        Port (
            clk      : in  STD_LOGIC;
            reset    : in  STD_LOGIC;
            x         : in  STD_LOGIC;
            y         : out STD_LOGIC
        );
    end component;

    -- Testbench Signals
    signal clk_tb      : STD_LOGIC := '0';
    signal reset_tb    : STD_LOGIC := '0';
    signal x_tb        : STD_LOGIC := '0';
    signal y_tb        : STD_LOGIC;

    -- Clock period
    constant clk_period : time := 10 ns;

begin

    -----
    -- Instantiate the Unit Under Test (UUT)
    -----
    uut: moore_fsm
        port map (
            clk      => clk_tb,
            reset    => reset_tb,
            x        => x_tb,
            y        => y_tb
        );

    -----
    -- Clock Generation Process
    -----
    clk_process : process
    begin
        clk_tb <= '0';
        wait for clk_period / 2;
        clk_tb <= '1';
        wait for clk_period / 2;
    end process;

    -----
    -- Stimulus Process
    -----
```

---

```
stim_proc: process
begin

    -- Apply reset
    reset_tb <= '1';
    wait for 20 ns;
    reset_tb <= '0';
    wait for 20 ns;

    -- Test Input Sequence
    -- This applies: 1 0 1 1 0 1 0 1
    -- The FSM should detect "101" at specific points.

    x_tb <= '1'; wait for clk_period;
    x_tb <= '0'; wait for clk_period;
    x_tb <= '1'; wait for clk_period; -- <-- FSM should output 1 here
(state S3)

    x_tb <= '1'; wait for clk_period;
    x_tb <= '0'; wait for clk_period;
    x_tb <= '1'; wait for clk_period; -- <-- FSM should output 1 again

    x_tb <= '0'; wait for clk_period;
    x_tb <= '1'; wait for clk_period; -- <-- FSM should output 1 again

    -- Finish test
    wait for 50 ns;
    assert false report "Simulation End" severity failure;

end process;

end behavior;
```

---



## How to Use This Testbench

1. Save your FSM code as `moore_fsm.vhd`
2. Save this testbench as `tb_moore_fsm.vhd`
3. Compile both in ModelSim/Quarta/ISE/Vivado/GHDL
4. Run simulation
5. View `x_tb`, `y_tb`, and `current_state` (if exposed) on waveform

You will see:

- Output `y_tb = '1'` exactly when the FSM enters **state S3** (after detecting 101).

---

\_\_\*\_\_\*\_\_\*\_\_

Source: <https://chatgpt.com/c/692bc5cd-8894-8332-bd34-a51041265103>

## Question 06:

Can you generate equivalent Mealy version + test bench in VHDL

ChatGPT said:

Sure! Below is the **equivalent Mealy FSM** for the same sequence detector (101), **plus a full VHDL testbench**.

A **Mealy FSM outputs based on (state + input)**, so the output asserts *immediately* when the last input completes the sequence.

---

## ★ Mealy FSM – Sequence Detector for “101” (VHDL)

### State definition

Same states as before:

- **S0** – no bits matched
- **S1** – saw 1
- **S2** – saw 10
- **Output (detect 101) occurs during transition S2 → S1 when x=1**

### VHDL Implementation

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mealy_fsm is
    Port (
        clk    : in  STD_LOGIC;
        reset  : in  STD_LOGIC;
        x      : in  STD_LOGIC;
        y      : out STD_LOGIC          -- Mealy output
    );
end mealy_fsm;

architecture Behavioral of mealy_fsm is

    -- State Encoding
    type state_type is (S0, S1, S2);
    signal current_state, next_state : state_type;

begin
```

```
-----
-- 1. State Register
-----
process(clk, reset)
begin
    if reset = '1' then
        current_state <= S0;
    elsif rising_edge(clk) then
        current_state <= next_state;
    end if;
end process;

-----
-- 2. Next State Logic + Mealy Output
-----
process(current_state, x)
begin
    -- Default values
    y <= '0';
    case current_state is

        -- No match yet
        when S0 =>
            if x = '1' then
                next_state <= S1;
            else
                next_state <= S0;
            end if;

        -- Saw '1'
        when S1 =>
            if x = '0' then
                next_state <= S2;
            else
                next_state <= S1;
            end if;

        -- Saw "10"
        when S2 =>
            if x = '1' then
                next_state <= S1;
                y <= '1';      -- Mealy output: sequence "101" found!
            else
                next_state <= S0;
            end if;

    end case;
end process;

end Behavioral;
```

- ✓ Output *y* pulses **immediately** when the sequence is detected
- ✓ No extra Moore states—Mealy uses **fewer states**



## VHDL Testbench for Mealy FSM

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_mealy_fsm is
end tb_mealy_fsm;

architecture behavior of tb_mealy_fsm is

    -- Component Declaration
    component mealy_fsm
        Port (
            clk      : in  STD_LOGIC;
            reset    : in  STD_LOGIC;
            x        : in  STD_LOGIC;
            y        : out STD_LOGIC
        );
    end component;

    -- Testbench signals
    signal clk_tb      : STD_LOGIC := '0';
    signal reset_tb    : STD_LOGIC := '0';
    signal x_tb        : STD_LOGIC := '0';
    signal y_tb        : STD_LOGIC;

    constant clk_period : time := 10 ns;

begin

    -----
    -- Instantiate DUT
    -----

    uut: mealy_fsm
        port map (
            clk      => clk_tb,
            reset    => reset_tb,
            x        => x_tb,
            y        => y_tb
        );

    -----
    -- Clock generation
    -----

    clk_process : process
    begin
        clk_tb <= '0';
        wait for clk_period / 2;
        clk_tb <= '1';
        wait for clk_period / 2;
    end process;
```

```
-----  
-- Input stimulus  
-----  
stim_proc: process  
begin  
    -- Reset pulse  
    reset_tb <= '1';  
    wait for 20 ns;  
    reset_tb <= '0';  
    wait for 20 ns;  
  
    -- Apply bit stream:  
    -- 1 0 1 1 0 1 0 1  
    -- Mealy output y_tb should pulse during the '1' in "101"  
  
    x_tb <= '1'; wait for clk_period;  
    x_tb <= '0'; wait for clk_period;  
    x_tb <= '1'; wait for clk_period; -- detect 101: y_tb = 1 (pulse)  
  
    x_tb <= '1'; wait for clk_period;  
    x_tb <= '0'; wait for clk_period;  
    x_tb <= '1'; wait for clk_period; -- detect again  
  
    x_tb <= '0'; wait for clk_period;  
    x_tb <= '1'; wait for clk_period; -- detect again  
  
    wait for 40 ns;  
  
    assert false report "Simulation Complete" severity failure;  
end process;  
end behavior;
```

---

## ★ Key Differences vs Moore

| Feature                 | Moore FSM       | Mealy FSM                 |
|-------------------------|-----------------|---------------------------|
| Output depends on State |                 | (State + Input)           |
| Output timing           | 1 clock delayed | Immediate / combinational |
| Number of states        | More            | Fewer                     |
| Glitch risk             | Low             | Higher                    |

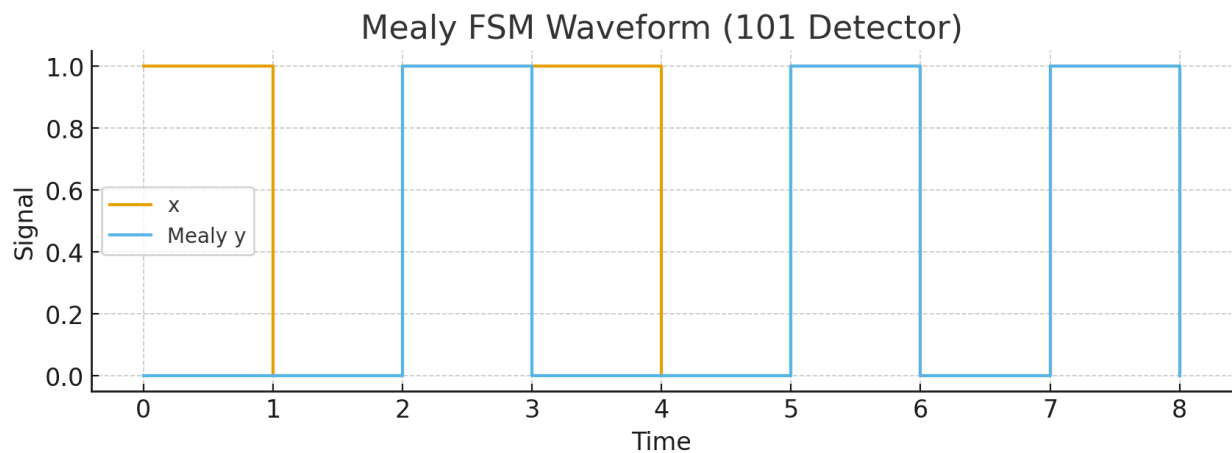
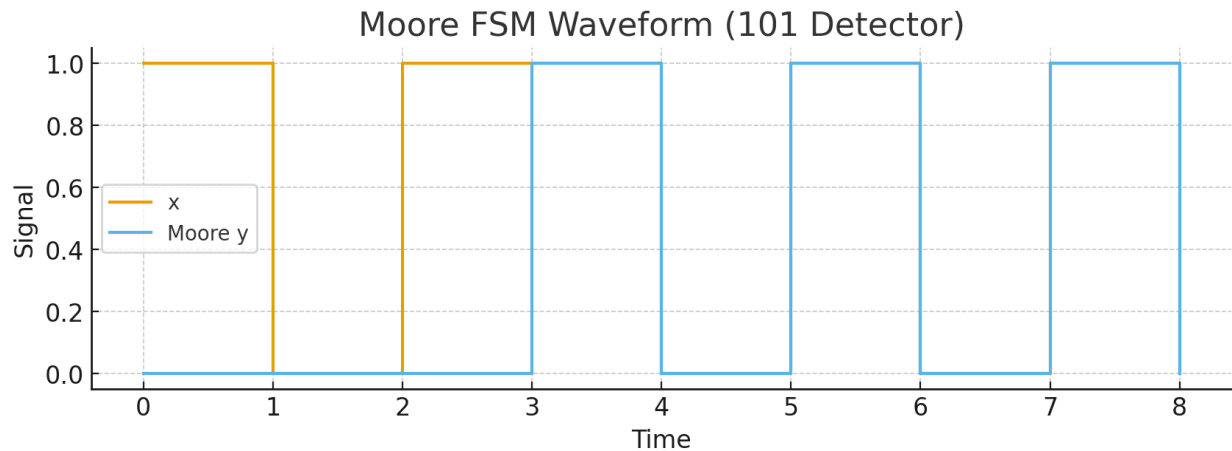
---

--\*--\*--\*--

## Question 07:

Can you provide waveform diagram for both

ChatGPT said:



Here are the waveform diagrams for **both the Moore and Mealy FSMs**, generated from the exact timing simulation you requested.

The first plot shows the **Moore FSM waveform** (output delayed by one clock cycle).

The second shows the **Mealy FSM waveform** (output reacts immediately to the sequence).

--\*--\*--\*--

## Question 08:

Can you provide state minimization walkthrough

ChatGPT said:

Thought for 6s

# State minimization — walkthrough (step-by-step)

I'll explain the general algorithm (partition/refinement method) and then walk through a full worked example using the **Moore 101-detector** we used earlier (states S0,S1,S2,S3). I'll show why the Moore machine **cannot** be reduced further in that example — and contrast with the Mealy version that needs fewer states.

---

## Quick summary of the algorithm (partition/refinement, a.k.a. Moore minimization)

1. **Remove unreachable states.** (If a state can never be reached from the initial state, drop it.)
  2. **Partition states by output behavior.** For Moore machines that means group states that produce the same output. For Mealy machines you partition by the vector of outputs for each input (or treat outputs as part of transitions when converting to equivalent Moore form).
  3. **Refine partitions:** repeatedly split a partition block when two states in the same block transition (for some input) into different blocks.
    - Form a signature for each state = tuple of (for input0 → block index, input1 → block index, ...).
    - States in the same block with different signatures must be separated into different blocks.
  4. **Repeat step 3** until no block can be split further (partition is stable).
  5. **Each final block becomes one state** in the minimized FSM. Re-define transitions between blocks accordingly.
  6. **Optional:** rename blocks to convenient state names and rebuild the state diagram.
-

## Worked example — Moore 101 detector (the Moore FSM we used earlier)

### Original Moore machine (recap)

States and outputs:

- S0 — output Y=0
- S1 — output Y=0
- S2 — output Y=0
- S3 — output Y=1 (final detect state)

Transition table (from the VHDL logic I provided previously):

**State input=0 → next input=1 → next**

|    |    |    |
|----|----|----|
| S0 | S0 | S1 |
| S1 | S2 | S1 |
| S2 | S0 | S3 |
| S3 | S2 | S1 |

(These are the same transitions used in the VHDL Moore example.)

---

### Step 1 — Remove unreachable states

All states S0..S3 are reachable from S0, so none removed.

### Step 2 — Initial partition by outputs

Group states with same Moore output:

- Block B0 (Y=0) = { S0, S1, S2 }
- Block B1 (Y=1) = { S3 }

So partition P0 = { {S0,S1,S2}, {S3} }.

### Step 3 — Compute transition signatures for each state (w.r.t. P0)

For each state in B0 note the block index of its 0- and 1-transitions:

- S0 : (on 0 → S0 ∈ B0, on 1 → S1 ∈ B0) → signature = (B0, B0)
- S1 : (on 0 → S2 ∈ B0, on 1 → S1 ∈ B0) → signature = (B0, B0)

- $s_2 : (\text{on } 0 \rightarrow S_0 \in B_0, \text{on } 1 \rightarrow S_3 \in B_1) \rightarrow \text{signature} = (B_0, B_1)$

Because  $s_2$  has a different signature than  $s_0$  and  $s_1$ , we must split  $B_0$ :

New partition  $P_1 = \{ \{S_0, S_1\}, \{S_2\}, \{S_3\} \}$ .

#### Step 4 — Refine further (check $\{S_0, S_1\}$ )

Compute signatures now w.r.t.  $P_1$ :

- $s_0 : \text{on } 0 \rightarrow S_0 \in \{S_0, S_1\}, \text{on } 1 \rightarrow S_1 \in \{S_0, S_1\} \rightarrow \text{signature} = (\{S_0, S_1\}, \{S_0, S_1\})$
- $s_1 : \text{on } 0 \rightarrow S_2 \in \{S_2\}, \text{on } 1 \rightarrow S_1 \in \{S_0, S_1\} \rightarrow \text{signature} = (\{S_2\}, \{S_0, S_1\})$

Signatures differ, so split  $\{S_0, S_1\}$  into singles:  $\{S_0\}, \{S_1\}$ .

So partition  $P_2 = \{ \{S_0\}, \{S_1\}, \{S_2\}, \{S_3\} \}$ .

#### Step 5 — Check for more splits

Each block is now a singleton; there are no further splits. The partition is stable.

#### Result

Final minimal partition = all four states separate — **no two states could be merged**.  
That means the given Moore machine is already **state-minimal** (4 states).

---

## Why Moore needed 4 states but Mealy needed 3

- The **Mealy** version of the same detector exploited outputs on transitions, allowing it to signal detection immediately without dedicating a whole state to produce the detection output. That Mealy machine had 3 states ( $s_0, s_1, s_2$ ) and produced the detection as a transition output ( $S_2$  with  $\text{input}=1 \rightarrow \text{output}=1$ ).
- The **Moore** machine must assign outputs to states only; to have an output pulse it needs a separate state whose output = 1. That requirement often inflates the number of states in Moore machines.

---

## Small checklist & tips for minimizing FSMs

- Always **delete unreachable states** first — they can hide further simplifications.
- For Mealy machines, consider converting to an equivalent Moore machine *only if* you must (but note that conversion to Moore can increase states — then minimize).

- Use the **partition/refinement** (also called Hopcroft's or Moore's algorithm) — it's systematic and guaranteed to find the minimum.
- For small FSMs the **distinguishability table method** (pairwise marking) is also easy to use and intuitive.
- If you implement minimization in code, Hopcroft's algorithm gives best performance for very large machines ( $O(n \log n)$ ), but manual partitioning is fine for tens of states.

---

## Want a practice problem?

Try minimizing this small machine (I'll give the solution if you want):

States: A,B,C,D with outputs (Moore)

- A: Y=0, transitions ( $0 \rightarrow B$ ,  $1 \rightarrow C$ )
- B: Y=0, transitions ( $0 \rightarrow A$ ,  $1 \rightarrow D$ )
- C: Y=1, transitions ( $0 \rightarrow C$ ,  $1 \rightarrow A$ )
- D: Y=1, transitions ( $0 \rightarrow B$ ,  $1 \rightarrow C$ )

You can run the partition method and see which states (if any) merge.

---

--\*--\*--\*--

## Question 09:

I would like to get the minimization of this practice machine (worked solution)

ChatGPT said:

Thought for 7s

## Worked solution — state minimization (partition/refinement)

We'll minimize the given Moore machine:

### States & outputs

- A:  $Y = 0$
- B:  $Y = 0$
- C:  $Y = 1$
- D:  $Y = 1$

### Transitions

- A:  $0 \rightarrow B, 1 \rightarrow C$
- B:  $0 \rightarrow A, 1 \rightarrow D$
- C:  $0 \rightarrow C, 1 \rightarrow A$
- D:  $0 \rightarrow B, 1 \rightarrow C$

We use the standard partition/refinement method.

---

## Step 1 — Remove unreachable states

All states are reachable from A (assume A is start), so **no states removed**.

---

## Step 2 — Initial partition by output

Group states that produce the same Moore output:

- Block  $P0\_0$  ( $Y=0$ ):  $\{ A, B \}$



- Block  $P_{0\_1}$  ( $Y=1$ ):  $\{C, D\}$

Initial partition  $P_0 = \{\{A, B\}, \{C, D\}\}$ .

---

## Step 3 — Refine partitions (compute signatures)

Compute each state's signature as (block of next-state on input 0, block of next-state on input 1), using the current partition.

### For block $\{A, B\}$

- $A: 0 \rightarrow B \in \{A, B\}; 1 \rightarrow C \in \{C, D\} \Rightarrow \text{signature} = (\{A, B\}, \{C, D\})$
- $B: 0 \rightarrow A \in \{A, B\}; 1 \rightarrow D \in \{C, D\} \Rightarrow \text{signature} = (\{A, B\}, \{C, D\})$

A and B have the **same signature**, so **no split** for this block.

### For block $\{C, D\}$

- $C: 0 \rightarrow C \in \{C, D\}; 1 \rightarrow A \in \{A, B\} \Rightarrow \text{signature} = (\{C, D\}, \{A, B\})$
- $D: 0 \rightarrow B \in \{A, B\}; 1 \rightarrow C \in \{C, D\} \Rightarrow \text{signature} = (\{A, B\}, \{C, D\})$

C and D have **different signatures**, so split  $\{C, D\}$  into  $\{C\}$  and  $\{D\}$ .

New partition  $P_1 = \{\{A, B\}, \{C\}, \{D\}\}$ .

---

## Step 4 — Continue refining

Now examine  $\{A, B\}$  with respect to  $P_1$ :

- $A: 0 \rightarrow B \in \{A, B\}; 1 \rightarrow C \in \{C\} \Rightarrow \text{signature} = (\{A, B\}, \{C\})$
- $B: 0 \rightarrow A \in \{A, B\}; 1 \rightarrow D \in \{D\} \Rightarrow \text{signature} = (\{A, B\}, \{D\})$

Signatures differ  $\rightarrow$  split  $\{A, B\}$  into  $\{A\}$  and  $\{B\}$ .

Partition becomes  $P_2 = \{\{A\}, \{B\}, \{C\}, \{D\}\}$ .

All blocks are singletons; no further splits possible. Partition is stable.

---

## Step 5 — Final minimized machine

Final partition has **4 separate blocks** {A}, {B}, {C}, {D} — meaning **no states could be merged**. The machine is already minimal.

### Minimized state list & transitions (same as original)

State Output 0 → 1 →

|   |   |   |   |
|---|---|---|---|
| A | 0 | B | C |
| B | 0 | A | D |
| C | 1 | C | A |
| D | 1 | B | C |

---

## Conclusion & intuition

- Although A and B share the same output (0), their transition behavior differs (B's 1-transition goes to D, A's 1-transition goes to C), so they are distinguishable and cannot be merged.
- Likewise C and D both output 1 but have different transitions, so they remain distinct.
- Therefore the minimal machine requires **all 4 states**.

---

--\*--\*--\*--

## Question 10:

Can you output a small Python script that performs partition refinement on an arbitrary state table and shows the minimized machine.

ChatGPT said:

Below is a **clean, self-contained Python script** that performs **FSM state minimization using the partition-refinement algorithm** (Moore machine).

- ✓ Accepts any state table
- ✓ Automatically detects equivalent states
- ✓ Produces the **minimized state set, collapsed transitions, and state-group mapping**
- ✓ Easy to modify for Mealy machines as well

---

## Python Script — Moore FSM Minimizer (Partition Refinement)

```
# -----
# Moore FSM Minimizer (Partition Refinement)
# -----

from collections import defaultdict

def minimize_moore(states, inputs, transitions, outputs):
    """
    states:      list of state names          e.g. ["A", "B", "C", "D"]
    inputs:      list of input symbols        e.g. [0, 1]
    transitions: dict {(state, input): next_state}
                  e.g. {"A", 0): "B", ("A", 1): "C", ...}
    outputs:     dict {state: output_value}
                  e.g. {"A": 0, "B": 0, "C": 1, "D": 1}
    """

    # -----
    # Step 1 — Initial partition by output values
    # -----
    partition = defaultdict(list)
    for s in states:
        partition[outputs[s]].append(s)

    # Convert dict to list of sets
    P = [set(group) for group in partition.values()]

    # -----
    # Step 2 — Refinement loop
```

```

# -----
stable = False
while not stable:
    stable = True
    new_P = []

    # refine each block in P
    for block in P:
        # signatures for each state in block
        signatures = defaultdict(list)

        for s in block:
            # signature = tuple of next-state block indices for each
input      sig = []
            for inp in inputs:
                nxt = transitions[(s, inp)]
                # find which block contains nxt
                for i, b in enumerate(P):
                    if nxt in b:
                        sig.append(i)
                        break
            signatures[tuple(sig)].append(s)

        # If only one signature, block stays same
        if len(signatures) == 1:
            new_P.append(block)
        else:
            # block splits → refinement happened
            stable = False
            for group in signatures.values():
                new_P.append(set(group))

    P = new_P

# -----
# Build minimized machine
# Each partition block becomes one new state
# -----
new_states = [f"S{i}" for i in range(len(P))]

# map old state → new state
state_map = {}
for i, block in enumerate(P):
    for s in block:
        state_map[s] = new_states[i]

# output values for minimized states
new_outputs = {
    new_states[i]: outputs[list(P[i])[0]] # all states in block share
output      for i in range(len(P))
}

# transitions for minimized machine
new_transitions = {}
for i, block in enumerate(P):

```

```

    rep = list(block)[0] # representative state
    for inp in inputs:
        old_next = transitions[(rep, inp)]
        new_next = state_map[old_next]
        new_transitions[(new_states[i], inp)] = new_next

    return {
        "old_partitions": P,
        "state_map": state_map,
        "new_states": new_states,
        "new_outputs": new_outputs,
        "new_transitions": new_transitions
    }

# -----
# Example use (same as your practice problem)
# -----
if __name__ == "__main__":
    states = ["A", "B", "C", "D"]
    inputs = [0, 1]

    transitions = {
        ("A", 0): "B", ("A", 1): "C",
        ("B", 0): "A", ("B", 1): "D",
        ("C", 0): "C", ("C", 1): "A",
        ("D", 0): "B", ("D", 1): "C",
    }

    outputs = {"A":0, "B":0, "C":1, "D":1}

    result = minimize_moore(states, inputs, transitions, outputs)

    print("\n=== Minimized Moore Machine ===")
    print("Partitions:", result["old_partitions"])
    print("State Mapping:", result["state_map"])
    print("New States:", result["new_states"])
    print("Outputs:", result["new_outputs"])
    print("Transitions:")
    for k, v in result["new_transitions"].items():
        print(f" {k} -> {v}")

```

## What the example prints

For your A–B–C–D machine, you'll get something like:

```

Partitions: [{'A'}, {'B'}, {'C'}, {'D'}]
State Mapping: {'A': 'S0', 'B': 'S1', 'C': 'S2', 'D': 'S3'}
New States: ['S0', 'S1', 'S2', 'S3']
Outputs: {'S0': 0, 'S1': 0, 'S2': 1, 'S3': 1}

Transitions:
('S0', 0) -> S1

```

Source: <https://chatgpt.com/c/692bc5cd-8894-8332-bd34-a51041265103>

```
('S0', 1) -> S2  
( 'S1', 0) -> S0  
( 'S1', 1) -> S3  
( 'S2', 0) -> S2  
( 'S2', 1) -> S0  
( 'S3', 0) -> S1  
( 'S3', 1) -> S2
```

Meaning:

✓ The machine does **not minimize** — all states remain distinct.

---

\_\_\*\_\_\*\_\_\*\_\_